

Microservices

Procedure to create a Spring boot project acting as Eureka Server.

Step.1: create spring boot project adding **Eureka Server** as dependency.

Step.2: add @EnableEurekaServer on the top of the main class.

Step.3: add the following entries in application.properties.

Application.properties

```
server.port=8761
eureka.client.register-with-eureka = false
eureka.client.fetch-registry = false
```

Step.4: Run the application.

Step.5: Test the application running or not in below url.

<http://localhost:8761>

Procedure to create a MS Development and Publishing it to Eureka Server

Step.1: Make Sure that spring boot project already developed and running as a Eureka Server.

Step.2: Create Spring starter project adding **spring web, EurekaDiscoveryClient** Dependencies

Step.3: Place @EnableDiscoveryClient Annotation on the top of main class.

Step.4: add the following entries in application.properties.

Application.properties

```
spring.application.name = PaymentService
server.port = 9001
eureka.client.service-url.default-zone = http://localhost:8761/eureka
```

Step.5: Develop one RestController representing the microservice.

Step.6: Run the MicroService Project.

(this process automatically registered MS to EurekaServer)

Step.7: Refresh the home page of eureka Server and observe the instance(project) came in that Section.

Step.8: Collect url from status section and modify it to generated the request to MS.

<http://localhost://9001/payment/bill>

Procedure to create a Api GateWay

Step.1: create spring boot project adding **GateWay, Spring Web** as dependency.

Step.2: add the following entries in application.properties.

Application.properties

```
spring.application.name=ApiGateWay  
server.port=8080
```

```
spring.cloud.gateway.mvc.routes[0].id=shoppingService  
spring.cloud.gateway.mvc.routes[0].uri=http://localhost:9081  
spring.cloud.gateway.mvc.routes[0].predicates[0]=Path=/shopping/*
```

```
spring.cloud.gateway.mvc.routes[1].id=billingService  
spring.cloud.gateway.mvc.routes[1].uri=http://localhost:8092  
spring.cloud.gateway.mvc.routes[1].predicates[0]=Path=/bill/*
```

Step.4: Run the application.

Step.5: Test the all applications accessing or not using below url.

<http://localhost:8080>

Procedure to Establish communication between projects using Feign Client.

Step.1: Develop Project as Eureka Server.

Step.2: Develop a Microservice projects which u have to communicate and deploy in Eureka Server.

Step.3: Select Consumer project.

=> Place **@EnableFeignClients** on the top of main class.

=> Take an interface supporting FeignClient code.

```
@FeignClient("billingService")
public interface IBillingConfig {
    @GetMapping("/bill/amt")
    public Double getBillAmount();
}
```

=> Develop the Consumer RestController injecting FeignClient related proxy object to consume the target/Producer Ms Service.

```
@RestController
@RequestMapping("/shopping")
public class ShoppingController {
    @Autowired
    private IBillingConfig conf;
    @GetMapping("/shop")
    public String shoppingMethods() {
        return "Bill Amount is :: "+conf.getBillAmount();
    }
}
```

Step.4: Execute the application.

1. Run Eureka Server
2. Run the Producer Ms.
3. Run the Consumer Ms.
4. Run the Api GateWay.
5. Go to the EurekaServer Home Page and Modify the consumer Service url as like as before.
6. Check the output both projects output came or not.

Procedure for using Github account maintained external config file through spring cloud Config Server.

Step.1: Create a application.properties(External Config file) in GitHub Account.

application.properties

dbuser = root
dbpwd = Ansol

Step.2.: create Eureka Server project.

Step.3: create a Spring cloud Configuration server Project(ConfigServer)

- => create a spring boot project by adding Config Server as Dependency.
- =>add `@EnableConfigServer` on main class.
- => add the following entries in application.properties file.

Application.properties

spring.application.name=ConfigServer
server.port=8888
spring.cloud.config.server.git.uri=<https://github.com/anil-1222222/Natraj.git>

Step.4: create Multiple Microservice Projects having the following dependencies

- a) Spring web b) EurekaDiscoveryClientetc c) Config Client(new)

MS1

- =>add `@EnableDiscoveryClient` on the main class.
- => add following entries in application.properties file

Application.properties

spring.application.name=shoppingService
server.port=9081
eureka.client.service-url.default-zone=<http://localhost:8761/eureka/>
spring.config.import=optional:configserver: (new)

- => Develop one RestController coonsuming the values of external config file(github application.properties file)

```
@RestController
@RequestMapping("/shopping")
public class ShoppingController {
    @Value("${dbuser}")
    private String user;
    @Value("${dbpwd}")
    private String pass;

    @GetMapping("/db")
    public String databaseProperties() {
        return "DbUser name is :: "+user +"===== "+ " DbPassword is :: "+pass;}
}
```

Similarly Establish **MS2**.

Step.5: Run all the applications. And check a outputs.

Procedure to work with @RefreshScope

Step.1: add spring boot actuator dependency to our MS Project.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Step.2: Activate the readymade actuator “refresh” in our MS Project.

– application.properties
management.endpoints.web.exposure.include=refresh

Step.3: place @RefreshScope on the top RestController of the Ms Project.

```
@RestController
@RequestMapping("/shopping")
@RefreshScope
public class ShoppingController {
```

Step.4: start Eureka Server, ConfigServer, Ms Projects and ApiGateWay as regular fashion.

Step.5: Test the url.

Step.6: Modify the entities in application.properties of github(External config file).

Step.7: Again test url (🔄🔄 Modifications does not reflect Surprisingly).

Step.8: Gather Endpoint details of “refresh” spring boot actuator and give POST mode request to it using postman tool.

Use this url to give post request : <http://localhost:9001/actuator/refresh>

Step.9: Test the url.

(🍌🍌 Changes will reflect)